Compilador fase 2: Análise Semântica e Geração de Código

Na **fase 1** do projeto, foi implementado um compilador que fazia a leitura de um **arquivo-fonte** na linguagem **Portugol** (baseada na **Linguagem Pascal)** e realizava a análise léxica e sintática do programa fonte. O objetivo desse projeto é implementar as fases de **Análise Semântica** e **Geração de Código Intermediário** do compilador, a implementação dessa fase será baseada na implementação realizada na **fase 1**, caso você não tenha implementado a **fase 1**, para essa fase, você **terá que implementar tanto a fase 1 quanto a fase 2**.

Análise Semântica

Na **Análise Semântica** o seu compilador deverá verificar se as construções sintáticas da fase anterior estão coerentes, o compilador implementado na fase anterior deve manter as funcionalidades de identificação de erros **léxicos** e **sintáticos,** e adicionalmente, emitir as mensagens de **erros semânticos**, caso ocorram.

Nessa fase, o Compilador realizará **verificações semânticas** para os identificadores reconhecidos em dois momentos:

- **Declaração**: Nas seções de declaração para: variáveis (local e global), funções, procedimentos e parâmetros o compilador **deve garantir que os identificadores usados sejam únicos em um determinado escopo**. Por escopo, considere que os identificadores declarados no programa principal têm escopo global e dentro de funções ou procedimentos escopo local. Caso ocorra uma repetição de identificador o **compilador deve ser finalizado, com uma mensagem explicativa, informando que ocorreu um erro semântico**. Para isso deve ser implementado uma **Tabela de Símbolos** que armazenará os identificadores declarados. Para cada identificador deve ser armazenado sua categoria (variável, parâmetro, função, procedimento). Caso o identificador seja uma variável ou parâmetro deve ser armazenado o endereço e escopo. Além das informações sugeridas você pode definir outras informações para que seja possível gerar código corretamente.

- **Corpo do programa:** Os identificadores podem ser referenciados no corpo do programa, como por exemplo numa expressão ou na chamada de uma função ou procedimento. Portanto toda vez que um identificador for referenciado no corpo de programa, o compilador deve verificar se o identificador foi declarado corretamente, ou seja, se está presente na Tabela de Símbolos, **caso não tenha sido declarado é gerado um erro semântico explicativo** e compilador é finalizado. Além disso deve ser checado se categoria do identificador é condizente para uma operação, por exemplo, não pode ser usado uma variável para uma chamada de procedimento.

Para armazenar as informações referentes aos identificadores, a **Tabela de Símbolos** deverá, **obrigatoriamente**, ser implementada utilizando a estrutura de dados **Tabela de Dispersão (HashTable)**, cada entrada da tabela é uma Lista Encadeada com nós (células) do tipo `TNo`. Abaixo segue a definição tipo `TNo` e do tipo `TTabelaSimbolos`. **Importante**: na sua implementação você deve incluir as definições abaixo no seu programa fonte.

```
#define PRIME_NUMER 211  // numero primo para reduzir colisoes

typedef struct _TNo{
   char ID[16];
   int endereço;
   // aqui você deve declarar os outros campos que julgar necessário para
   // armazenar os identificadores

   struct _TNo *prox;
}TNo;

typedef struct {
    TNo *entradas[PRIME_NUMER];
}TTabelaSimbolos;
```

Note que a **Tabela de Símbolos** possui **211** entradas (`PRIME_NUMER`), para determinar em qual entrada será armazenado ser armazenado o identificador você deve utilizar a função `int hashMack(char * s)`, que recebe uma *string* como entrada e devolve a entrada (um inteiro) na **Tabela de Símbolos**, a função está implementada no

arquivo objeto **`hash_mack.o`** (para Linux). Para testar a função e entender como a função funciona, veja o programa fonte **`testa_hash_mack.c`**.
Para simplificar a análise semântica e geração de código intermediário, o compilador não precisará realizar a distinção entre expressões **inteiras** e **lógicas**, assim, para os testes do compilador, só teremos variáveis e expressões do tipo inteiro, também, não teremos construções que misturam os dois tipos, por exemplo: **`25+(x>y)`**.

Geração de Código Intermediário

A Geração das instruções da MEPA será realizada nas funções mutuamente recursivas implementadas no analisador sintático, para tanto basta **imprimir as instruções da MEPA** nas mesmas funções que fazem análise sintática e semântica do compilador.

Execução do Compilador – fase 2

Na saída do **compilador**, caso o programa não tenha erro léxico sintático ou semântica, deve ser impresso, no monitor do computador, a **tradução do código fonte para linguagem MEPA e a Tabela de Símbolos com os identificadores declarados.** Para cada identificador armazenado na tabela de símbolos deve ser impresso a sua entrada na tabela, ID (lexema), caso seja uma variável ou parâmetro também deve ser informado seu endereço.

Se o código fonte tiver erros, deve ser gerada uma mensagem informativa imprimindo o número da linha no arquivo fonte, onde o primeiro erro ocorre, e qual o tipo (**léxico**, **sintático** ou **semântico**) do erro identificado.

Não é necessário imprimir os átomos reconhecidos na fase léxica.

Observações importantes:

O programa deve ser devidamente documentado e poderá ser desenvolvido em grupo de **até dois alunos**. É **imprescindível** que os nomes dos integrantes do grupo sejam mencionados no início do arquivo fonte do trabalho.
Fica **terminantemente proibido** o uso de ferramentas de **Inteligência Artificial**, como o ChatGPT, para a **geração automática do código do projeto**. Qualquer tentativa de burlar esta restrição será considerada uma infração disciplinar, conforme o **COMUNICADO DA FCI de 05/02/2025**.

Para esclarecer alguma dúvida quanto a entrega do projeto, poderá ser REQUISITADO UMA ENTREVISTA COM OS INTEGRANTES que implementaram o projeto, com intuito de validar se as regras acima foram todas atendidas. É importante que os integrantes do grupo saibam responder questões sobre qualquer parte do projeto.

Critérios de Avaliação do Trabalho:

- O trabalho deve ser desenvolvido na **linguagem C** e será testado usando o compilador GCC instalado na versão Ubuntu-22.04, caso não tenha o Ubuntu instalado consulte o documento **WSL_Windows_Ubuntu_VSCode.pdf** disponível no Moodle.
- O trabalho deve ser desenvolvido na **linguagem C** e será testado usando o compilador **gcc** do Linux, para compilar seu programa com o seguinte comando abaixo, considere que o programa fonte do seu compilador seja `compilador.c`:

  ```
  gcc -Wall -Wno-unused-result -g -Og compilador.c hash_mack.o -o compilador
  ```

  Para executar use:
  ```
  ./compilador <nome do arquivo de entrada>
  ```

- Caso o programa não compile ou não execute **será atribuída a nota 0 ao trabalho.**
- Caso programa apresentarem ***warning*** durante a compilação, será descontado 1.0 (um ponto) por ***warning*** relatado.
- O programa não pode conter erro de acesso memória, caso tenha a nota será reduzida à metade. Para testar a memória do seu programa use o **valgrind** conforme descrito abaixo:

  ```
  valgrind --leak-check=yes ./compilador <nome do arquivo de entrada>
  ```

- O Código intermediário gerado deve, **obrigatoriamente**, ser executado pela máquina virtual da **`mepa`** disponível em https://github.com/flubacheski/Compiladores/tree/main/MEPA_Interpretador

- Entregue um arquivo **`readme.txt`** explicando como compilar e rodar o seu trabalho, informe também até qual parte do trabalho foi concluída, além de relatar quaisquer *bugs* ou erros identificados na sua implementação. No arquivo você pode compartilhar alguma decisão de *design* e implementação que que foram tomadas durante o desenvolvimento.